Decentralized Patient Record Storage

Problem Statement:

The patient record is one of the important assets that is currently centralized, maintained, and managed by hospitals.

Some countries have moved to Electronic Health Record (EHR), and many countries still follow the old method to store the patient and medical details. Around 80 to 90% of the hospitals in the United States have adopted basic EHR systems, which have a lot of problems related to privacy, security, and ownership of these health records.

Even after moving to EHR, the patient record is completely owned and controlled by hospitals, and being the owner of that record, a patient does not have access to the data and has to always reach out to hospitals to get it.

Data consolidation is one of the biggest challenges as every hospital stores these EHRs on their server. The patients need to access different EHRs to consolidate their complete medical history.

Since hospitals own the EHRs, they have full control and can manipulate the data at any moment. They can even sell the data to research organization and earn a good amount of money.

Along with the patient record, doctor authenticity is another challenge that is faced as there is no direct way to check if the doctor is real or not.

Solution:

Blockchain is a technology designed to manage patient data that has the potential to support transparency and accountability. A blockchain is a ledger of transactions where an identical copy is visible to all the members of a computer network. Network members validate the data entered into the ledger, and once entered, the data is immutable.

With the help of Blockchain technology, a solution can be developed to store the EHR of any patient in a distributed and decentralized network. With blockchain solutions, patients can own their data and can approve hospitals, doctors, and research organizations to access their data.

Roles:

In this solution, we are going to focus on two roles:

- Patient
- Doctor

A patient can do the following functions:

- Register to the portal
- Add new diseases
- View their records
- Update their data

A doctor can do the following things:

- Register to the portal
- Access and add the records of their patients
- Prescribe medicine to a patient

Find the code base of the program (EHR.sol).

```
pragma solidity >=0.8.0;

contract EHR {

address public Admin;

//Constructor to assign address to Admin

constructor ()
{
 Admin=msg.sender;
}
//Modifier Definition both for Admin and Patient
modifier OnlyAdmin()
{
   require(msg.sender==Admin, "You don't have access to this function");
   _; }

modifier OnlyRegPatient
{
require (Patient[msg.sender].isRegistered,"Patient not found..! You have to register to add a disease");
_;
```

```
}


// Define struct for Doctor as Doc
struct Doc {
   string Doc_Name;
   string Doc_Qual;
   string Work_Address;
   address Doc_Address;
   bool isRegistered;
}

// Define struct for Patient as Pat
struct Pat {

  string Pat_Name;
  uint8 Pat_Age;
  address Pat_Address;
  string[] Pat_Disease;
  bool   isRegistered;
  uint[] Pat_PrescribedMed;
}

// Define struct for Medicine as Med

struct Med {
   uint Med_Id;
   string Med_Name;
   string Med_ExpiryDate;
   string Med_Dose;
   string Med_Price;
   bool isRegistered;

}

// Define the mapping
 mapping ( address => Doc) Doctor;
 mapping ( address => Pat) Patient;
 mapping ( uint => Med) Medicine;

// Function to Register Patient
```

```
function RegisterPatient(string memory _PatName,uint8 _PAge) public {
require (!Patient[msg.sender].isRegistered,"You are alreadey Registered (Patient)");
Patient[msg.sender].Pat_Name=_PatName;
Patient[msg.sender].Pat_Age=_PAge;
Patient[msg.sender].Pat_Address=msg.sender;
Patient[msg.sender].isRegistered=true;
}
```

//function to Add Medicine

```
function AddMedicine(uint _Id,string memory _MedName,string memory _Exp,string memory
_Dos,string memory _P) public {
   require (!Medicine[_Id].isRegistered,"This Medicine Already Registred");

   Medicine[_Id]=Med(_Id,_MedName,_Exp,_Dos,_P,true);

}
```

//Function to Add Disease

```
function AddNewDiseasebyPatient(string memory _PatientDisease) public OnlyRegPatient {

// require (Patient[msg.sender].isRegistered,"Patient Not found, Need to Register Patient");
   Patient[msg.sender].Pat_Disease.push(_PatientDisease);
}
```

// Function to Register Doctor

```
function RegisterDoctor(string memory _DocName,string memory _DocQual, string memory
_WAddress, address _DocAddress) public OnlyAdmin
{
   require (!Doctor[_DocAddress].isRegistered,"You are alreadey Registered (Doctor)");
   Doctor[_DocAddress]=Doc(_DocName,_DocQual,_WAddress,_DocAddress,true) ;
  }
```

// Function to Prescribe Medicine

```
function PrescribeMedicine(uint _MedId, address _PatAddress) public {
   Patient[_PatAddress].Pat_PrescribedMed.push(_MedId);
}
```

```
// Function to Update Age

function UpdateAge(uint8 _NewAge) public OnlyRegPatient {

   Patient[msg.sender].Pat_Age=_NewAge;
}

// Function to view Patient Record
function ViewPatientRecord() public OnlyRegPatient view returns (uint age, string memory name,string[]
memory disease ,address patientAddress)
{

   age=Patient[msg.sender].Pat_Age;
   name=Patient[msg.sender].Pat_Name;
   disease= Patient[msg.sender].Pat_Disease;
   patientAddress=Patient[msg.sender].Pat_Address;

}

// Function to view Prescribed Medicine
function ViewPrescribedMedicines(address Paddress) public view returns (uint[]memory)
{
return (Patient[Paddress].Pat_PrescribedMed);
}

// Function to view Medicine
function viewMedicine(uint MedId) public view returns (string memory, string memory, string memory,
string memory) {
   return (Medicine[MedId].Med_Name, Medicine[MedId].Med_ExpiryDate,
Medicine[MedId].Med_Dose, Medicine[MedId].Med_Price);
}

// Function to view Patient ByAddress
function ViewPatientByAddress(address EthAddress) public view returns (string memory, uint8 ,
string[]memory, uint[]memory){
   return(Patient[EthAddress].Pat_Name, Patient[EthAddress].Pat_Age,
Patient[EthAddress].Pat_Disease, Patient[EthAddress].Pat_PrescribedMed);
}

// Function to view Doctor
```

```
function ViewDoctor(address GiveMeEthAddress) public view returns (string memory, string
memory,string memory)
{
   return(Doctor[GiveMeEthAddress].Doc_Name, Doctor[GiveMeEthAddress].Doc_Qual,
Doctor[GiveMeEthAddress].Work_Address);
}

}
```

Technologies Used:

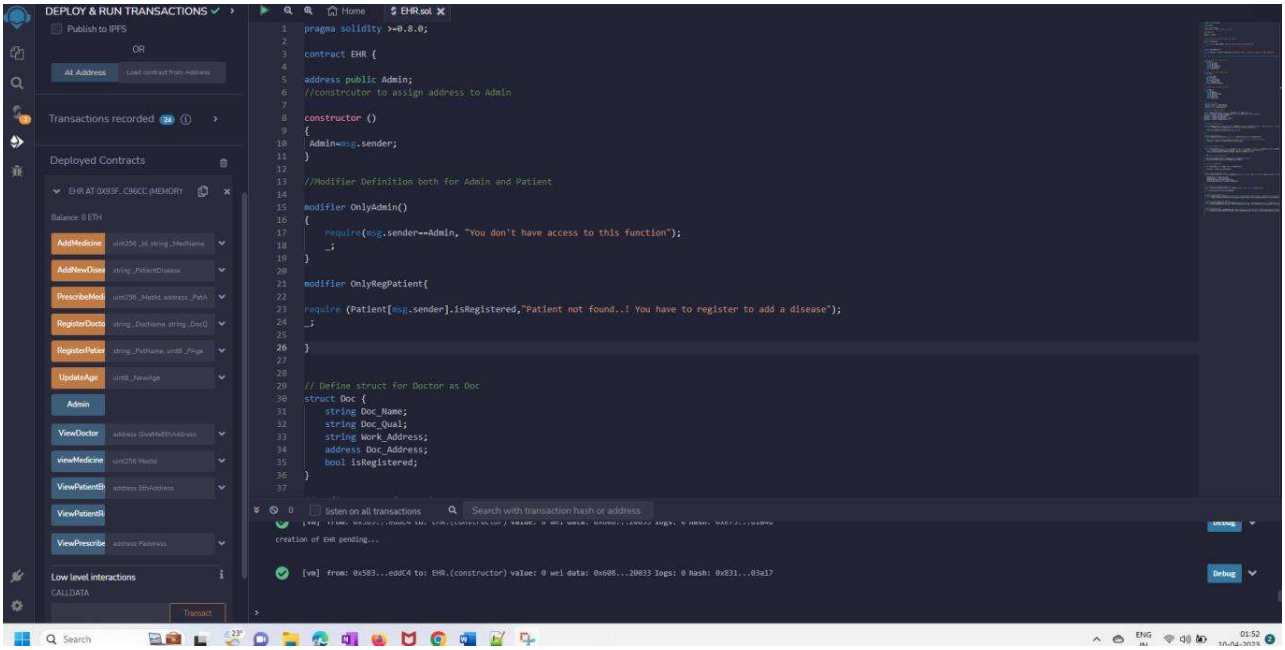
1. Smart Contract development: Solidity
2. IDE Tool: Remix
3. Blockchain: Ethereum
4. Server: Ganache Blockchain

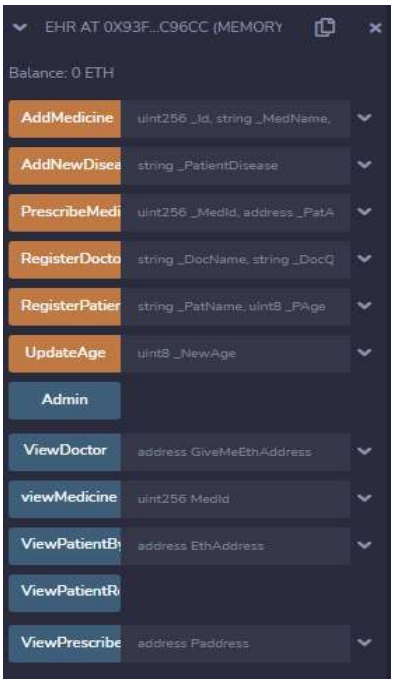
Project development guidelines:

- The project is delivered with sprint. Following are user stories.
  - Find the requirement from EHR attributes point of view
  - Define the roles like Doctor, Patient and other attributes like Medicine etc.
  - Identify the environment for development and deployment.
  - Develop the MVP
  - Test the functionalities and sign-off the MVP.
- Below link stores the codebase in GitHub.
  - tapasn2603/blockchainEHR: Ethereum codebase for Decentralized Employee Health Record (github.com)

Project outputs:

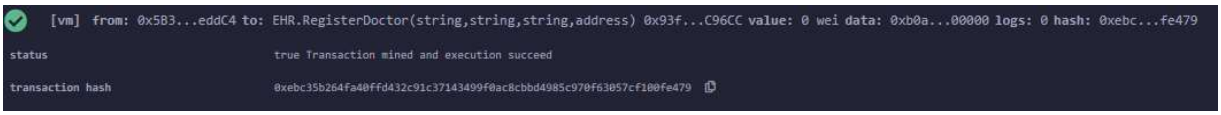
Post deployment, following are the screens and functionalities tested.



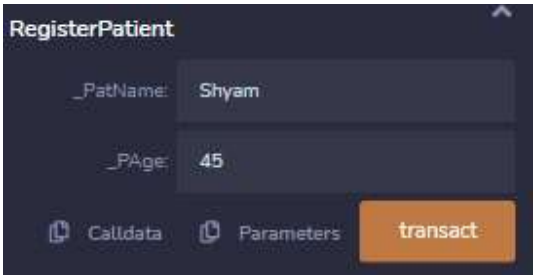


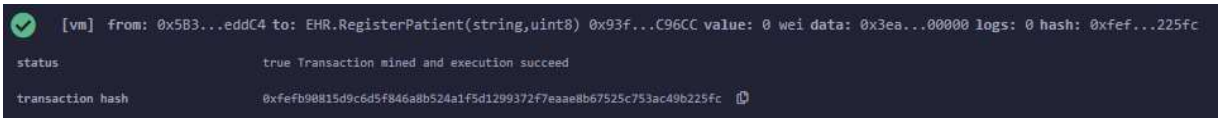
Function Details:

Register a new doctor using the below function:

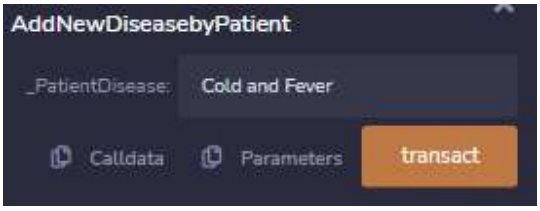


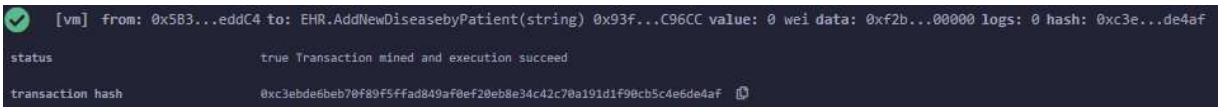
Add a new patient:



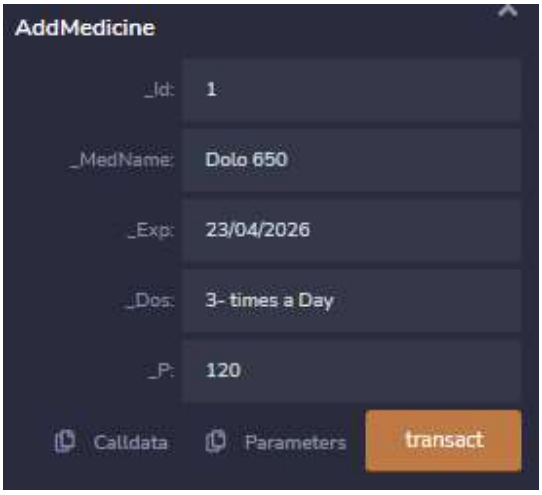


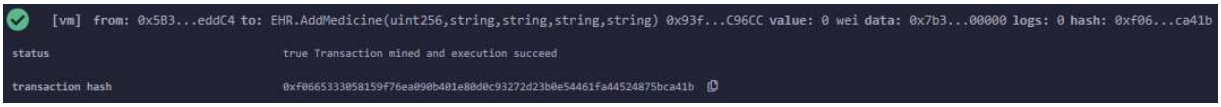
Add a new disease for the patient:



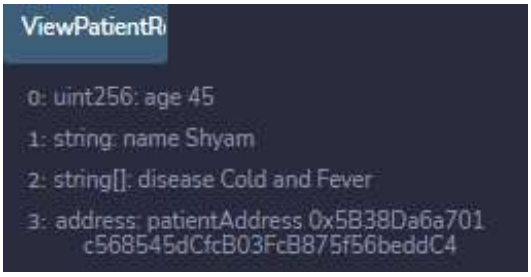


Add medicine to the ledger:

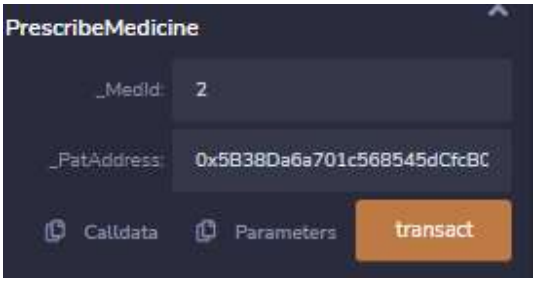


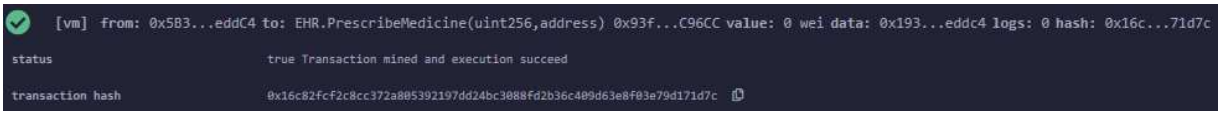


View Patient Record:

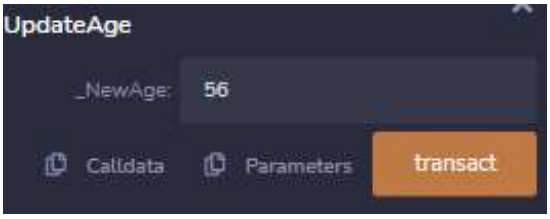


Prescribe medicine to the patient:

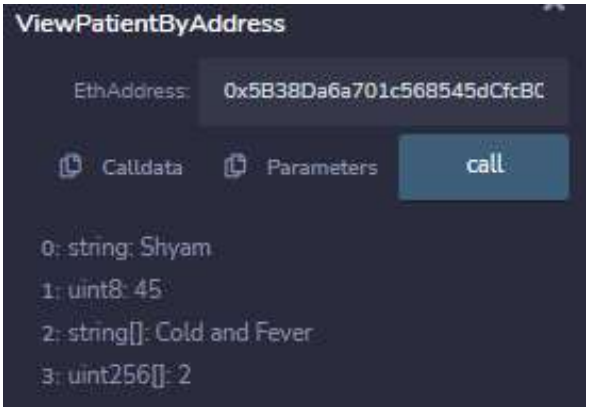




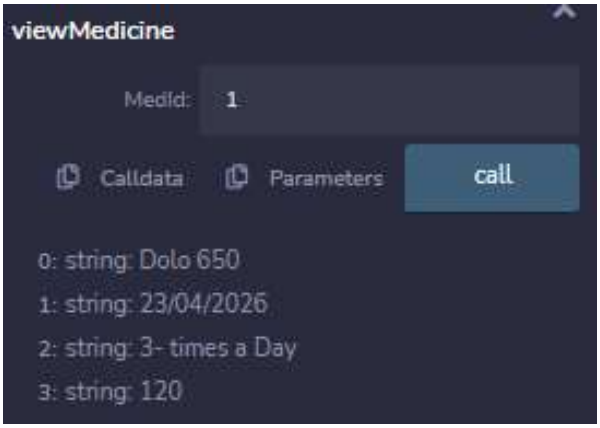
Update the age of the patient:



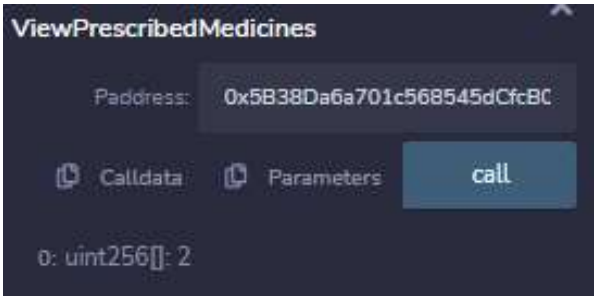
View patient data by address:



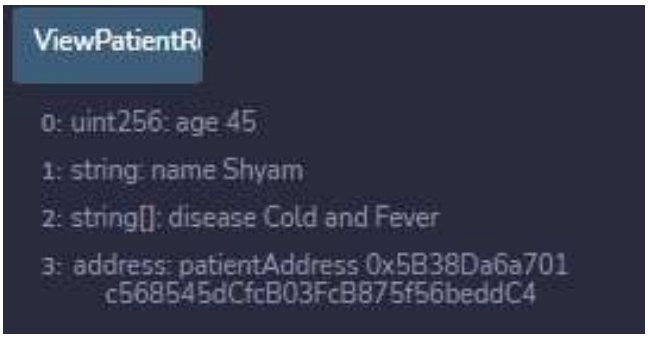
View medicine details:



View prescribed medicine:



View patient data by doctor:



View doctor details: